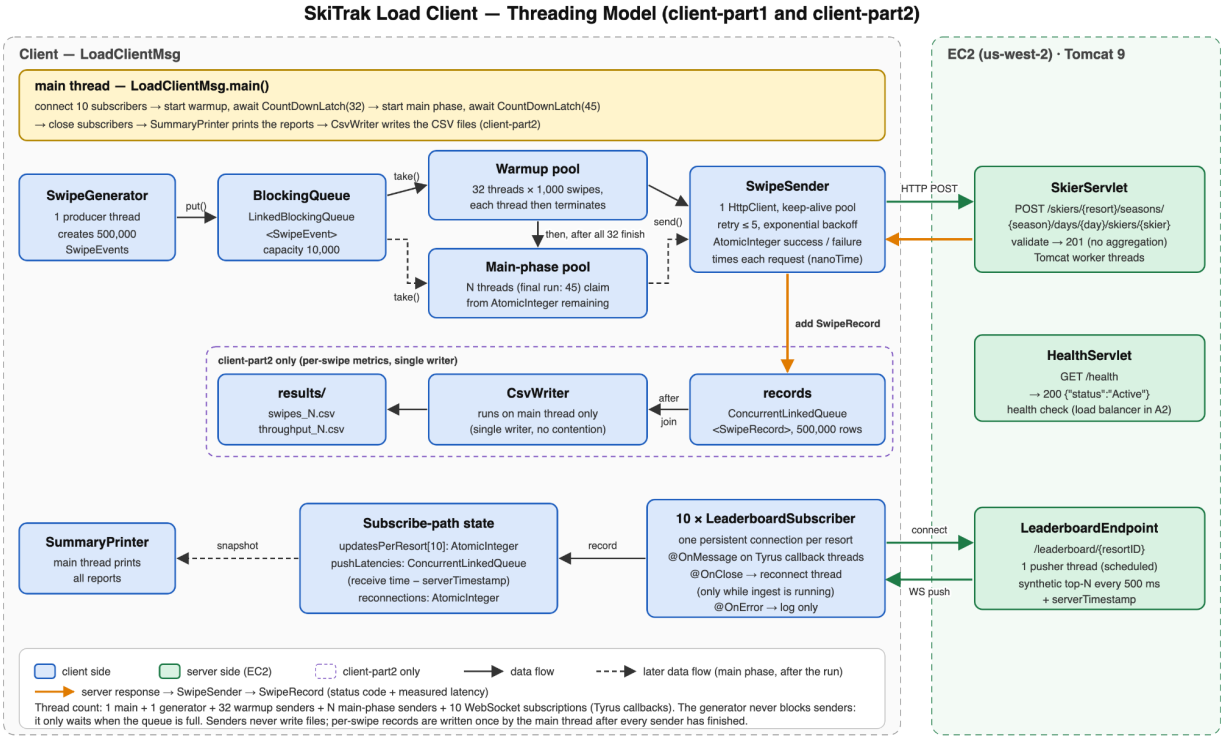
1.Design Doc

1.1 Architecture diagram

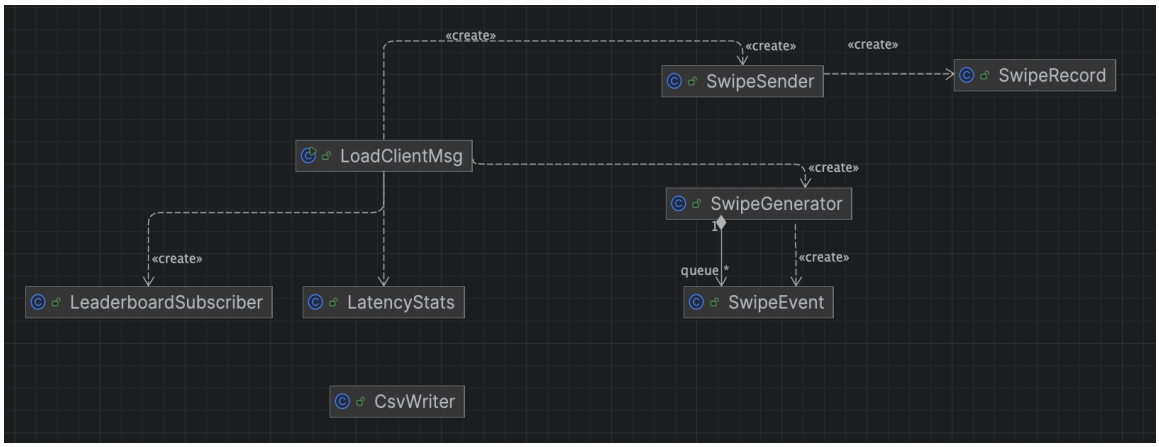


1. Swipe generation and ingest. SwipeGenerator puts swipes into the BlockingQueue; the warmup pool and then the main-phase pool take them and pass them to SwipeSender, which sends each one to SkierServlet as an HTTP POST.

2. Response recording. SkierServlet returns a response to SwipeSender, which turns it into a SwipeRecord and adds it to records; after all senders finish, CsvWriter writes records to results/.

3. Leaderboard push. LeaderboardEndpoint pushes leaderboards over WebSocket to the LeaderboardSubscribers, which store them in the subscribe-path state for SummaryPrinter to report.

1.2 Major Classes UML Diagram



1.3 websocket connection management

The client keeps a fixed subscription pool of 10 persistent WebSocket connections, one per resort (/leaderboard/{resortID}), all created from one shared WebSocketContainer. All 10 connections open before the warmup phase and close after the main phase ends, so they stay connected for the whole ingest run. Each LeaderboardSubscriber uses three callbacks:

@OnMessage: Handling pushes. It runs on Tyrus callback threads, records the receive time before parsing serverTimestamp, and updates thread-safe counters (AtomicInteger, ConcurrentLinkedQueue).

@OnClose: For reconnection. If the connection drops while ingest is still running, it starts a separate reconnect thread, so the callback thread is never blocked. That thread retries up to 5 times, waiting 1–5 s between attempts.

@OnError: Logging only. @OnClose always follows an error, so one drop triggers exactly one reconnect.

The main thread is responsible for active and shutdown. It sets an `ingestDone` flag for safely closing the session

1.4 Little's Law

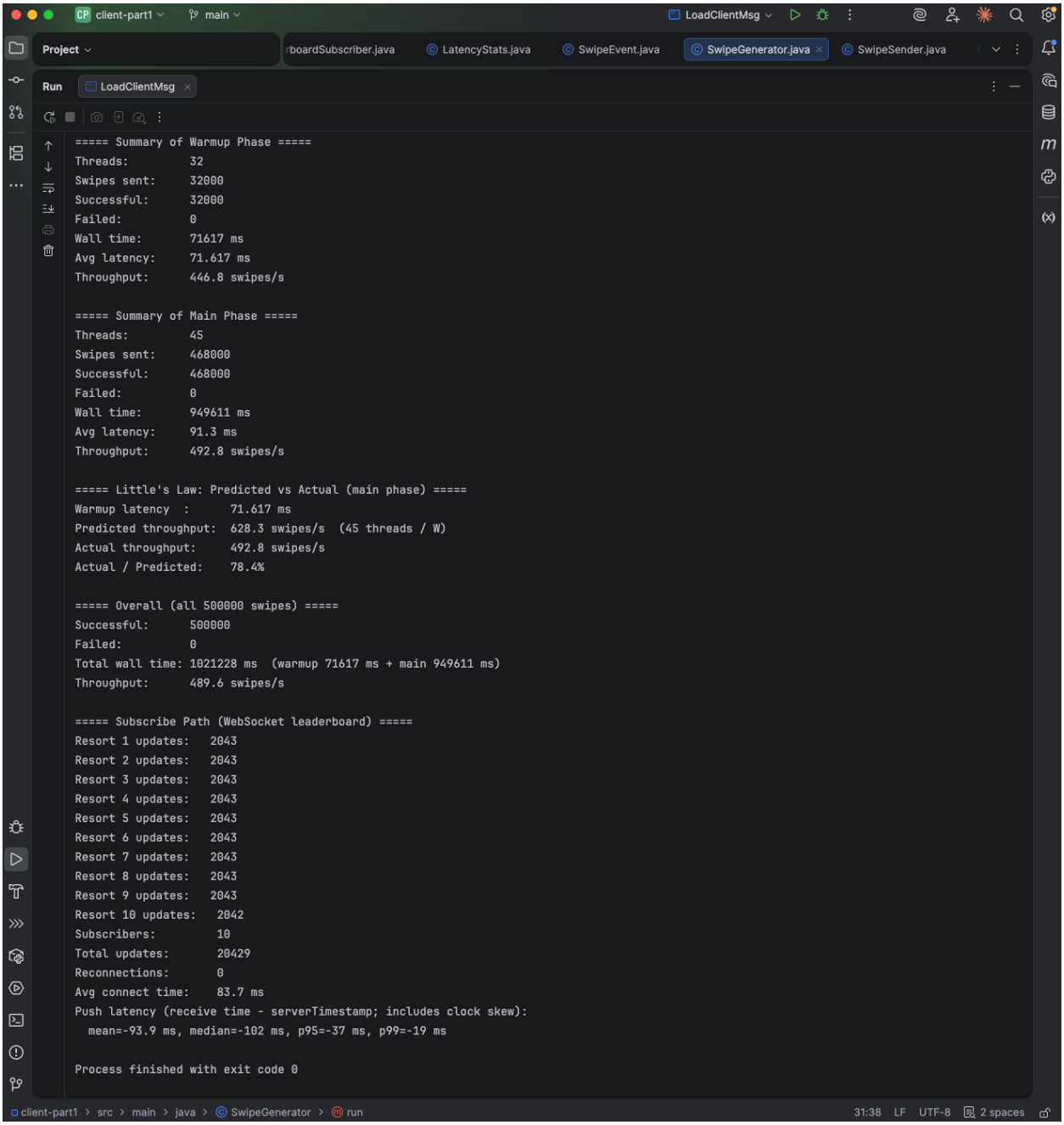
Using Little's Law, **λ = L / W**, where L is the number of concurrent sender threads and W is the average response time. We take W from the warmup phase and use it to predict the main-phase throughput.

Threads (L)	Warmup W	Predicted λ	Actual λ	Actual / Predicted	Main-phase W
45	71.6 ms	628.3 /s	492.8 /s	78.4%	91.3 ms
55	71.0 ms	774.3 /s	456.6 /s	59.0%	120.5 ms
64	61.6 ms	1039.1 /s	528.9 /s	50.9%	121.0 ms

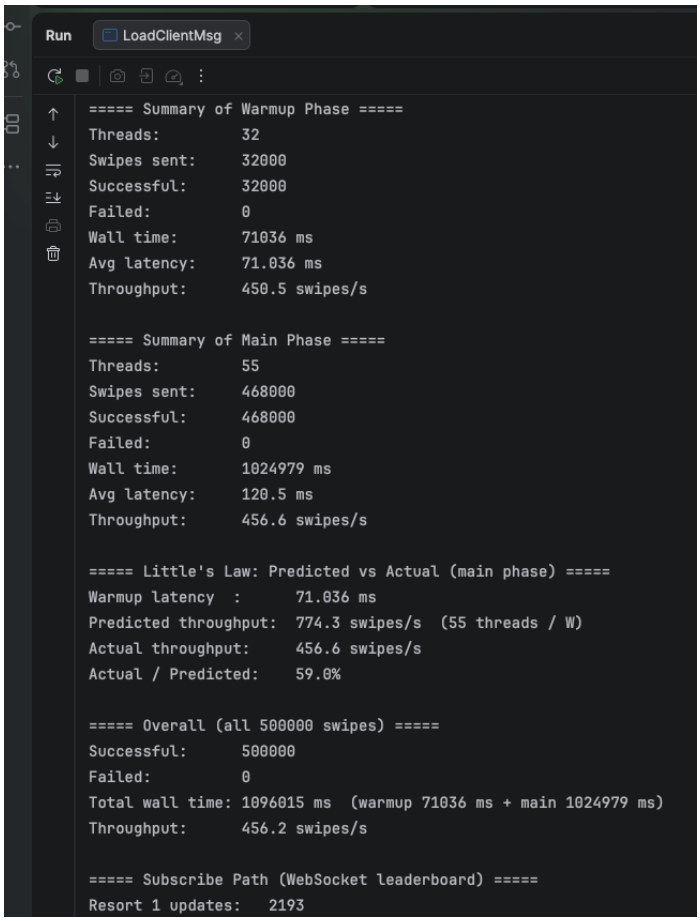
Gap. The prediction assumes W stays constant as L grows. In the main phase, W instead rose to 91–121 ms. λ × W still equals L in every run, so all threads stay busy and the gap comes entirely from the higher latency. Both EC2 and the client were mostly idle (Section 2.1), so the extra waiting happens in the network path. Throughput levels off at about 490–530 swipes/s, and 45 threads already reaches that level.

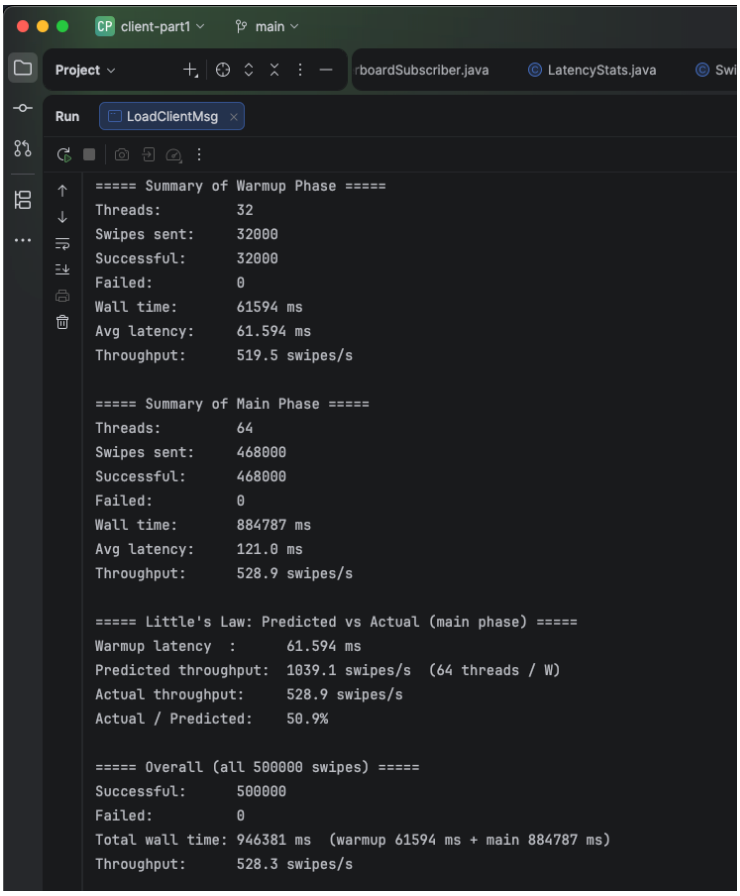
2.Test Results

2.1 Part-1 output



Based on 45 thread （Optimal Choice）

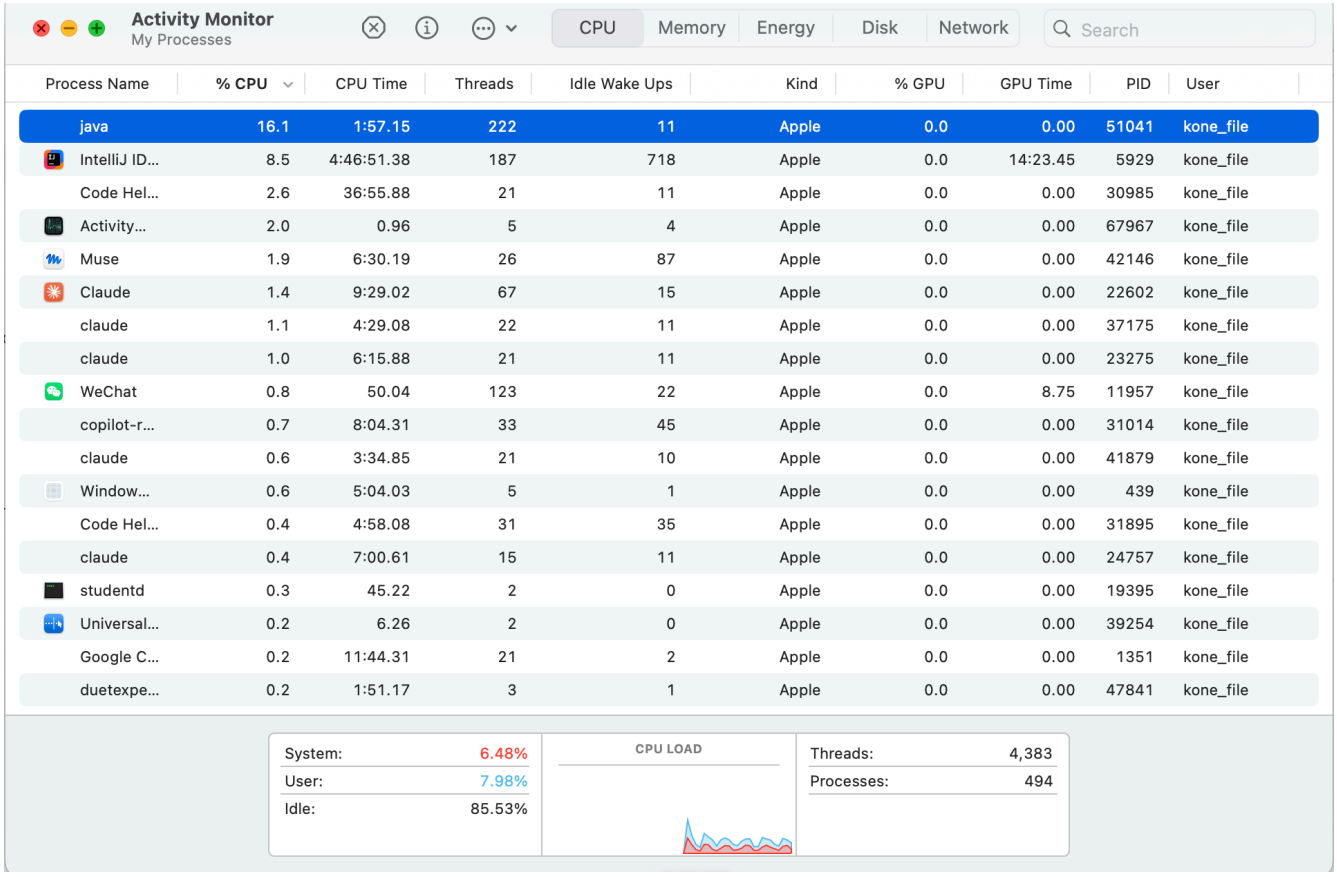




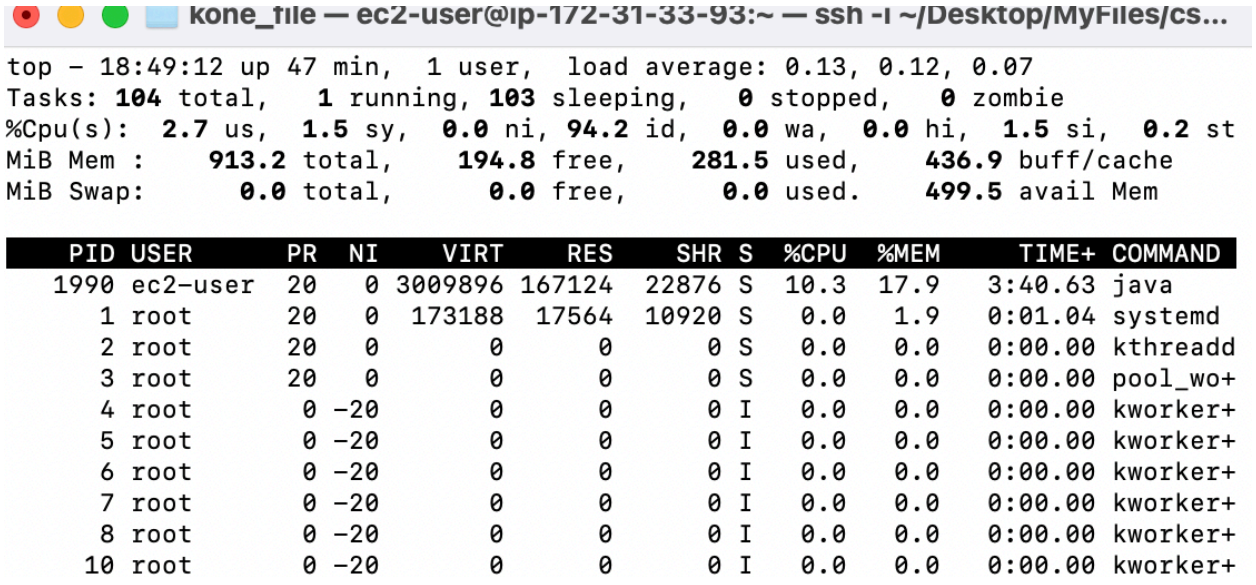
Based on 55 and 64 threads

Increasing concurrency beyond 45 threads did not increase throughput. Relative to each run's own warmup, throughput improved by 10% at 45 threads but by only 1–2% at 55 and 64 threads, while latency kept rising.

Since both the local machine and the EC2 instance had low CPU and memory usage, the bottleneck is the network between them, so adding more threads only adds waiting time. We therefore chose 45 threads as the optimal main-phase concurrency.



Monitor of Local Machine

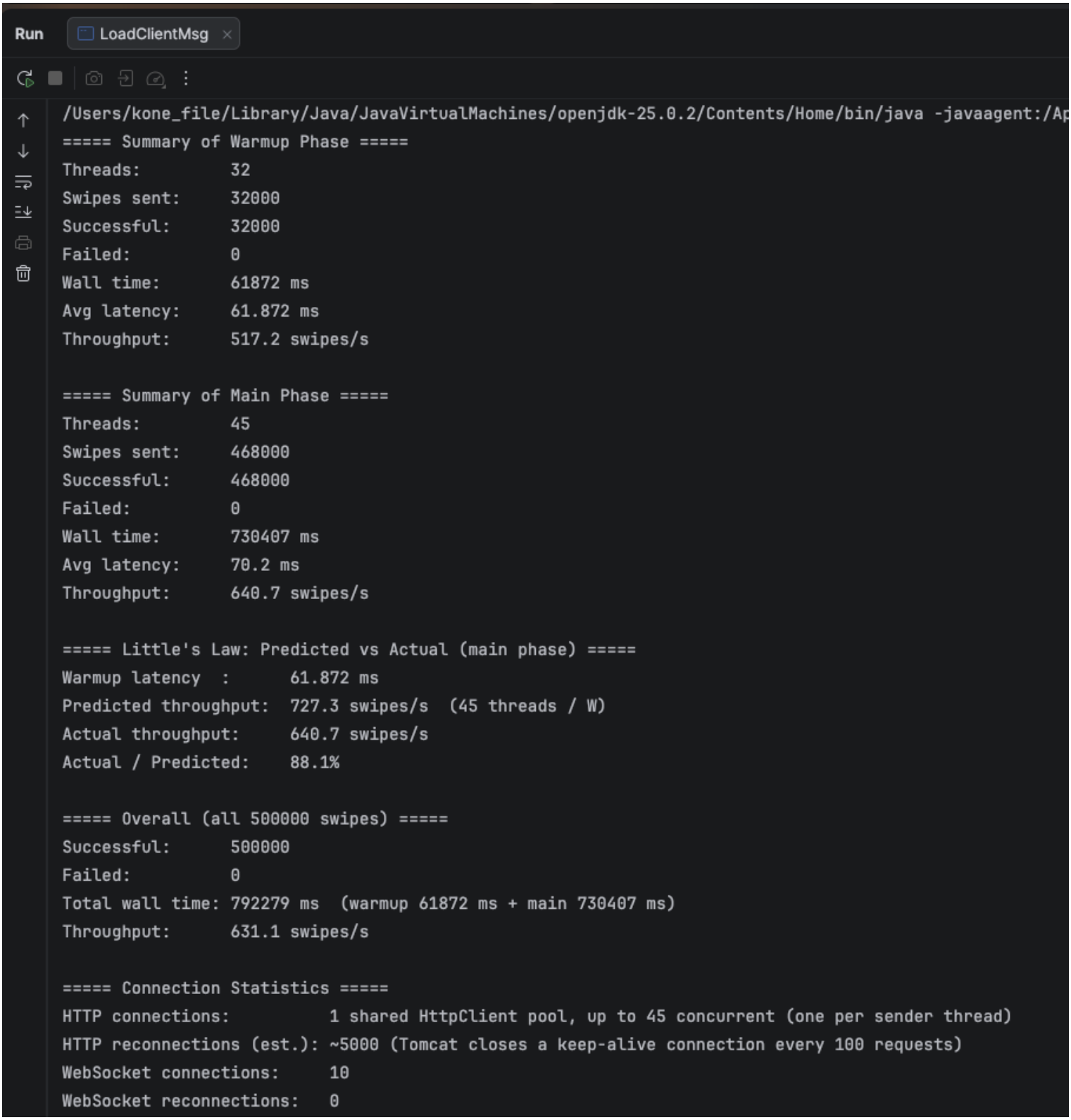


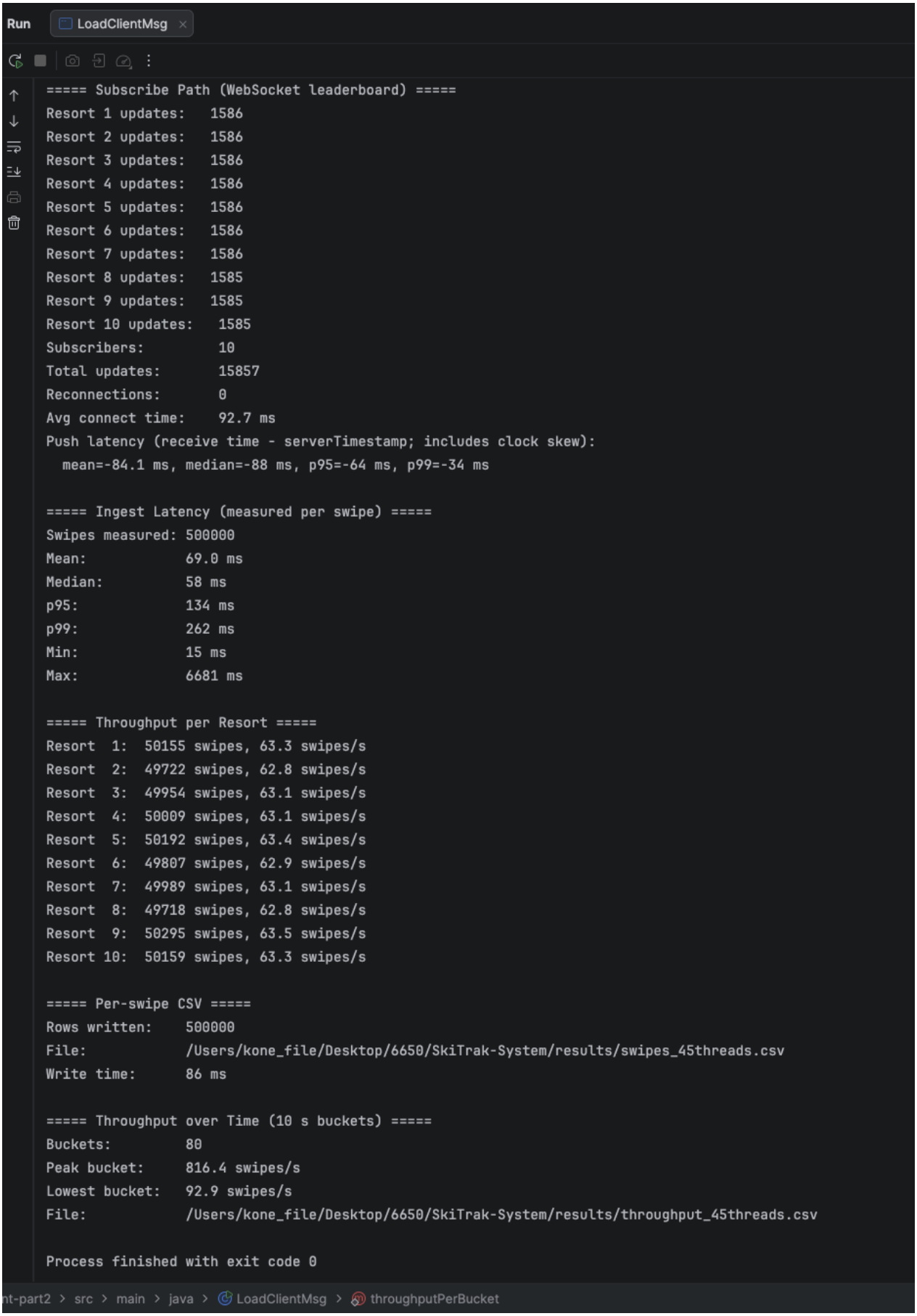
CPU usage of EC2

Little's Law shows that beyond 45 threads, latency grows almost exactly in proportion to the thread count, so each extra request is waiting in a queue somewhere. The resource measurements rule out every candidate except the network.

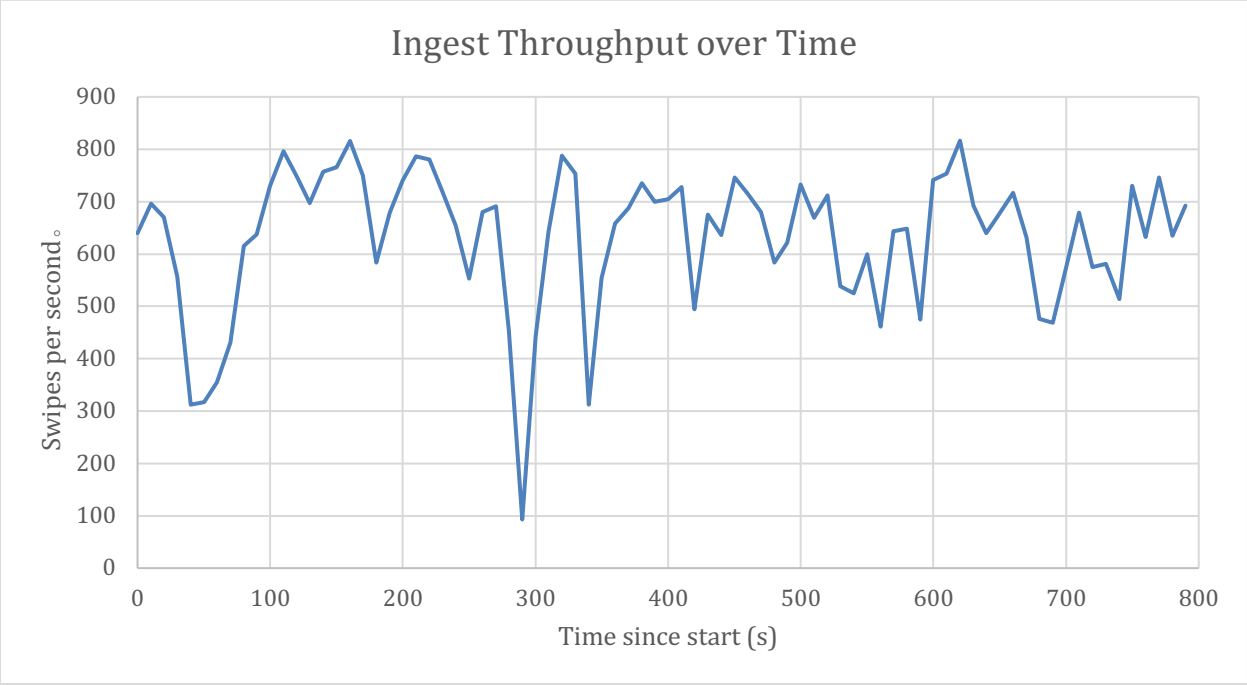
- **Server CPU:** EC2 is 94.2% idle, and its 1-minute load average of 0.13 is far below its single vCPU, so no requests are waiting for CPU.
- **CPU-credit throttling:** steal time is only 0.2%, so the t2.micro is not being throttled.
- **Client:** the client JVM uses 16.1% of one core, and the Mac is 85.5% idle, so the client is not compute-bound.

2.2 Part-2 Output




Run
LoadClientMsg
===== Subscribe Path (WebSocket leaderboard) =====
Resort 1 updates: 1586
Resort 2 updates: 1586
Resort 3 updates: 1586
Resort 4 updates: 1586
Resort 5 updates: 1586
Resort 6 updates: 1586
Resort 7 updates: 1586
Resort 8 updates: 1585
Resort 9 updates: 1585
Resort 10 updates: 1585
Subscribers: 10
Total updates: 15857
Reconnections: 0
Avg connect time: 92.7 ms
Push latency (receive time - serverTimestamp; includes clock skew):
mean=-84.1 ms, median=-88 ms, p95=-64 ms, p99=-34 ms
===== Ingest Latency (measured per swipe) =====
Swipes measured: 500000
Mean: 69.0 ms
Median: 58 ms
p95: 134 ms
p99: 262 ms
Min: 15 ms
Max: 6681 ms
===== Throughput per Resort =====
Resort 1: 50155 swipes, 63.3 swipes/s
Resort 2: 49722 swipes, 62.8 swipes/s
Resort 3: 49954 swipes, 63.1 swipes/s
Resort 4: 50009 swipes, 63.1 swipes/s
Resort 5: 50192 swipes, 63.4 swipes/s
Resort 6: 49807 swipes, 62.9 swipes/s
Resort 7: 49989 swipes, 63.1 swipes/s
Resort 8: 49718 swipes, 62.8 swipes/s
Resort 9: 50295 swipes, 63.5 swipes/s
Resort 10: 50159 swipes, 63.3 swipes/s
===== Per-swipe CSV =====
Rows written: 500000
File: /Users/kone_file/Desktop/6650/SkiTrak-System/results/swipes_45threads.csv
Write time: 86 ms
===== Throughput over Time (10 s buckets) =====
Buckets: 80
Peak bucket: 816.4 swipes/s
Lowest bucket: 92.9 swipes/s
File: /Users/kone_file/Desktop/6650/SkiTrak-System/results/throughput_45threads.csv
Process finished with exit code 0
nt-part2 > src > main > java > LoadClientMsg > throughputPerBucket

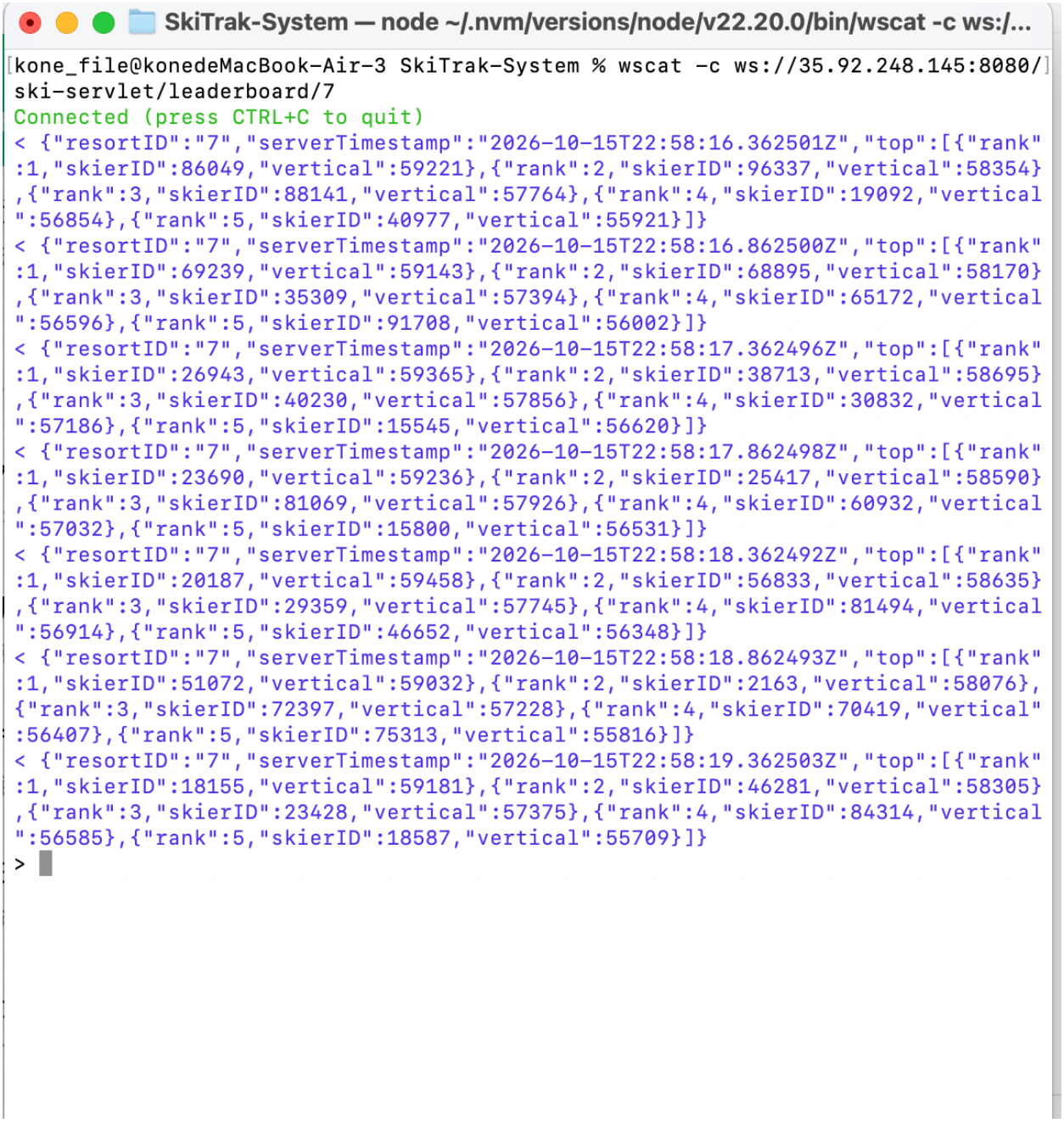
2.3 Throughput-over-time chart



2.4 EC2 Deployment



2.5 Receiving synthetic push



3.GitHub URL

https://github.com/Kone-H/SkiTrak-System